1 - Pages, Presentation & CRUD

So we have our data ready and we have some of the functionality of fetching and creating jobs with a title and description. Now we want to start building the presentation of the website and showing all the job listing data as well as creating new jobs with all the fields and form values. We'll create a nice looking job card component. We also want to be able to update and delete jobs. So at the end of this section, we'll have full CRUD functionaliy and it will look good. We're also going to learn about things like flash messages and we'll create an alert component to say when a job has been created or updated.

I'm also going to introduce Alpine.js in this section, which is a lightweight JavaScript library that gives you custom attributes you can use in your Blade templates to make your UI interactive. We're going to be able to dismiss the alert messages. We'll also re-factor the mobile menu button to use Alpine rather than vanilla Javascript. Alpine is very popular in the Laravel world. So let's get into it.

2 - Jobs Page & Card Component

Let's start off with the /jobs page. Right now, it just shows a list of titles. We want to have a nice detailed card for each job listing. We will also add a search form at the top of the page but that functionality will be added later.

We will create a new Blade component for the job card. This will allow us to reuse the card in multiple places. We will also update the jobs page to use this new component.

Open the `resources/views/components/job-card.blade.php` file. The HTML for this can be found in the `_theme_files/jobs.html`.

We need the grid wrapping markup and the card markup. Let's add the grid wrapper first. In the `resources/views/jobs/index.php` file, add the following code:

```
<x-layout>
  <x-slot:pageTitle>All Jobs</x-slot:pageTitle>
  <h1 class="text-2xl">{{ $title }}</h1>
  <div class="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
    @forelse($jobs as $job)
    <div>{{ $job->title }}</div>
    @empty
    <p>No jobs found</p>
    @endforelse
  </div>
</x-layout>
```

Instead of an unordered list, we will use a grid with 3 columns. This will allow us to have 3 jobs per row. Right now it is only showing the title. We want to replace that `<div>{{ $job->title }}</div>` with the job card component.

Job Card Component

Let's create a new Blade component for the job card. Run the following command to create a new Blade component:

```
php artisan make:component JobCard
```

Open the `resources/views/components/job-card.blade.php` file. and add the following code for now:

```
@props(['job'])
<div class="rounded-lg shadow-md bg-white p-4">{{$job->title}}</div>
```

This is a very basic card that takes in the job as a prop. We will add more details to it later. For now, let's update the `resources/views/jobs/index.php` file to use this new component.

Replace the `<div>{{ $job->title }}</div>` with the following code:

```
<x-job-card :job="$job" />
```

Now you should see the titles with the background color and rounded corners.

Let's add the rest of the job details and markup:

```
@props(['job'])
<div class="rounded-lg shadow-md bg-white p-4">
  <div class="flex items-center space-between gap-4">
    <img
      src="/images/{{ $job->company_logo }}"
      alt="{{$job->company_name}}"
      class="w-14"
    />
    <div>
      <h2 class="text-xl font-semibold">{{ $job->title }}</h2>
      <p class="text-sm text-gray-500">{{ $job->job_type }}</p>
    </div>
  </div>
  <p class="text-gray-700 text-lg mt-2">
    {{ Str::limit($job->description, 100) }}
  </p>
  <ul class="my-4 bg-gray-100 p-4 rounded">
    <li class="mb-2">
      <strong>Salary:</strong> ${{ number_format($job->salary) }}
    </li>
    <li class="mb-2">
      <strong>Location:</strong> {{ $job->city }}, {{ $job->state }} @if
      ($job->remote)
      <span class="text-xs bg-green-500 text-white rounded-full px-2 py-1 ml-2">
        Remote
```

```
        </span>
        @else
        <span class="text-xs bg-red-500 text-white rounded-full px-2 py-1 ml-2">
          On-site
        </span>
        @endif
      </li>
      <li class="mb-2">
            <strong>Tags:</strong> {{ucwords(str_replace(',', ', ', $job->tags))}}
        </li>
    </ul>
    <a
      href="{{ route('jobs.show', $job->id) }}"
      class="block w-full text-center px-5 py-2.5 shadow-sm rounded border text-base
font-medium text-indigo-700 bg-indigo-100 hover:bg-indigo-200"
    >
      Details
    </a>
  </div>
```

We have just changed the hardcoded values to use the job properties. We have also added a link to the job details page. For the description, I used the `Str::limit()` function to limit the description to 100 characters. Other than that, everything is pretty straightforward.

Logo

The company logo is not showing because they are not in the project. Open the `_theme_files/images` folder and copy the entire `logos` folder to the `public/images/logos` folder. This will allow the images to be accessible from the browser and you should now see the logos.

We will handle the search form later on as well as pagination.

In the next lesson, we will handle displaying the job listings on the homepage.

3 - Homepage Jobs

We want to show the latest 6 job listings on the homepage. This means we need to go into the `app/Http/Controllers/HomeController.php` file and make some changes.

Right now, the `index` method just returns a view. We want to return the view with the latest 6 job listings. We can do this by using the `Job` model. We can get the latest 6 job listings by using the `latest` method and then limiting the results to 6.

Bring in the `Job` model by adding the following line to the top of the file:

```
use App\Models\Job;
```

Here is the updated `index` method:

```
public function index(): View
{
    $jobs = Job::latest()->limit(6)->get();

    return view('pages.home')->with('jobs', $jobs);
}
```

Update the Home View

Now open the `resources/views/pages/home.blade.php` file and add the wrapper div and loop over the jobs and output the job card component. We will also add the "Recent Jobs" heading and a link to the `/jobs` page. Here is the updated code:

```
<x-layout>
  <h2 class="text-center text-3xl mb-4 font-bold border border-gray-300 p-3">
    Recent Jobs
  </h2>
  <div class="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
    @forelse($jobs as $job)
    <x-job-card :job="$job" />
    @empty
    <p>No jobs found</p>
    @endforelse
  </div>
  <a href="{{ route('jobs.index') }}" class="block text-xl text-center">
    <i class="fa fa-arrow-alt-circle-right"></i> Show All Jobs
  </a>
  <x-bottom-banner />
</x-layout>
```

Alright, our homepage looks great. Let's move on to the job details page.

4 - Job Details Page

Now we want to do the single job listing page. There is a lot of markup here, so we will be copying and pasting some of it.

Open the `resources/views/jobs/show.blade.php` file. This is our view and we already have access to the job object. We can use this to display the job details.

Open the `_theme_files/job-details.html` file. We are going to copy section by section as opposed to copying the whole file.

Let's start by typing out the grid container. Make the page look like this:

```
<x-layout>
  <div class="grid grid-cols-1 md:grid-cols-4 gap-6"></div>
</x-layout>
```

Now in the `_theme_files/job-details.html` file, let's copy and paste the job details column `<section>` inside the grid container. It looks like this:

```
<section class="md:col-span-3">
  <div class="rounded-lg shadow-md bg-white p-3">
    <div class="flex justify-between items-center">
      <a class="block p-4 text-blue-700" href="/jobs.html">
        <i class="fa fa-arrow-alt-circle-left"></i>
        Back To Listings
      </a>
      <div class="flex space-x-3 ml-4">
        <a
          href="/edit"
          class="px-4 py-2 bg-blue-500 hover:bg-blue-600 text-white rounded"
          >Edit</a
        >
        <!-- Delete Form -->
        <form method="POST">
          <button
            type="submit"
            class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
          >
            Delete
          </button>
        </form>
        <!-- End Delete Form -->
      </div>
    </div>
    <div class="p-4">
      <h2 class="text-xl font-semibold">Software Engineer</h2>
      <p class="text-gray-700 text-lg mt-2">
        As a Software Engineer at Algorix, you will be responsible for
        designing, developing, and maintaining high-quality software
        applications. You will work closely with cross-functional teams to
        deliver scalable and efficient solutions that meet business needs. The
        role involves writing clean, maintainable code, participating in code
        reviews, and staying current with industry trends to ensure our
        technology stack remains cutting-edge.
      </p>
      <ul class="my-4 bg-gray-100 p-4">
        <li class="mb-2"><strong>Job Type:</strong> Full Time</li>
        <li class="mb-2"><strong>Remote:</strong> No</li>
        <li class="mb-2"><strong>Salary:</strong> $80,000</li>
        <li class="mb-2">
          <strong>Site Location:</strong> New York, NY
          <span
            class="text-xs bg-blue-500 text-white rounded-full px-2 py-1 ml-2"
```

```
            >Local</span
          >
        </li>
        <li class="mb-2">
          <strong>Tags:</strong>
          <span>Development</span>,
          <span>Coding</span>
        </li>
      </ul>
    </div>
  </div>

  <div class="container mx-auto p-4">
    <h2 class="text-xl font-semibold mb-4">Job Details</h2>
    <div class="rounded-lg shadow-md bg-white p-4">
      <h3 class="text-lg font-semibold mb-2 text-blue-500">Job Requirements</h3>
      <p>
        Bachelors degree in Computer Science or related field, 3+ years of
        software development experience
      </p>
      <h3 class="text-lg font-semibold mt-4 mb-2 text-blue-500">Benefits</h3>
      <p>Healthcare, 401(k) matching, flexible work hours</p>
    </div>
    <p class="my-5">
      Put "Job Application" as the subject of your email and attach your resume.
    </p>
    <a
      href="mailto:manager@company.com"
      class="block w-full text-center px-5 py-2.5 shadow-sm rounded border text-
base font-medium cursor-pointer text-indigo-700 bg-indigo-100 hover:bg-indigo-200"
    >
      Apply Now
    </a>
  </div>

  <div class="bg-white p-6 rounded-lg shadow-md mt-6">
    <div id="map"></div>
  </div>
</section>
```

We are going to ignore the delete button and the map for now. We will come back to those later.

Edit Button

We have our resource routes for jobs and we have a controller method that just outputs a string for now, but let's make the edit button link to the edit page. Change the href of the edit button to use the route helper:

```
<a
  href="{{ route('jobs.edit', $job->id) }}"
  class="px-4 py-2 bg-blue-500 hover:bg-blue-600 text-white rounded"
```

```
    >Edit</a
  >
```

It takes in the controller method name and the id of the job. This will take us to the edit page.

Back Button

Let's change the href of the back link:

```
<a class="block p-4 text-blue-700" href="{{ route('jobs.index') }}">
  <i class="fa fa-arrow-alt-circle-left"></i>
  Back To Listings
</a>
```

This will take us back to the jobs index page.

Job Details

Now let's add the job details like the title, description, etc. Here is the the whole page:

```
<x-layout>
  <div class="grid grid-cols-1 md:grid-cols-4 gap-6">
    <section class="md:col-span-3">
      <div class="rounded-lg shadow-md bg-white p-3">
        <div class="flex justify-between items-center">
          <a class="block p-4 text-blue-700" href="{{ route('jobs.index') }}">
            <i class="fa fa-arrow-alt-circle-left"></i>
            Back To Listings
          </a>
          <div class="flex space-x-3 ml-4">
            <a
              href="/edit"
              class="px-4 py-2 bg-blue-500 hover:bg-blue-600 text-white rounded"
              >Edit</a
            >
            <!-- Delete Form -->
            <form method="POST">
              <button
                type="submit"
                class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
              >
                Delete
              </button>
            </form>
            <!-- End Delete Form -->
          </div>
        </div>
        <div class="p-4">
          <h2 class="text-xl font-semibold">{{$job->title}}</h2>
```

```
          <p class="text-gray-700 text-lg mt-2">{{$job->description}}</p>
          <ul class="my-4 bg-gray-100 p-4">
            <li class="mb-2"><strong>Job Type:</strong> {{$job->job_type}}</li>
            <li class="mb-2">
              <strong>Remote:</strong> {{$job->remote ? 'Yes' : 'No'}}
            </li>
            <li class="mb-2">
              <strong>Salary:</strong> ${{ number_format($job->salary) }}
            </li>
            <li class="mb-2">
              <strong>Site Location:</strong> {{$job->city}}, {{$job->state}}
            </li>
            <li class="mb-2">
              <strong>Tags:</strong>
              {{ ucwords(str_replace(',', ', ', $job->tags)) }}
            </li>
          </ul>
        </div>
      </div>

      <div class="container mx-auto p-4">
        <h2 class="text-xl font-semibold mb-4">Job Details</h2>
        <div class="rounded-lg shadow-md bg-white p-4">
          <h3 class="text-lg font-semibold mb-2 text-blue-500">
            Job Requirements
          </h3>
          <p>{{$job->requirements}}</p>
          <h3 class="text-lg font-semibold mt-4 mb-2 text-blue-500">
            Benefits
          </h3>
          <p>{{$job->benefits}}</p>
        </div>
        <p class="my-5">
          Put "Job Application" as the subject of your email and attach your
          resume.
        </p>
        <a
          href="mailto:{{$job->contact_email}}"
          class="block w-full text-center px-5 py-2.5 shadow-sm rounded border
text-base font-medium cursor-pointer text-indigo-700 bg-indigo-100 hover:bg-
indigo-200"
        >
          Apply Now
        </a>
      </div>

      <div class="bg-white p-6 rounded-lg shadow-md mt-6">
        <div id="map"></div>
      </div>
    </section>
  </div>
</x-layout>
```

Everything is pretty straightforward. We are just outputting the job details from the job object. We are also formatting the salary with the number_format() function so that it shows a comma between every 3 digits. We are also using the ucwords() function to capitalize the first letter of each word in the tags. We used the str_replace() function to replace the commas with commas and spaces.

Sidebar

Now let's do the sidebar, which will contain the company logo, company name and description as well as a button to bookmark the job.

In the `_theme_files/job-details.html` file, copy the sidebar section:

```
<aside class="bg-white rounded-lg shadow-md p-3">
  <h3 class="text-xl text-center mb-4 font-bold">Company Info</h3>
  <img
    src="images/logo-algorix.png"
    alt="Ad"
    class="w-full rounded-lg mb-4 m-auto"
  />
  <h4 class="text-lg font-bold">Algorix</h4>
  <p class="text-gray-700 text-lg my-3">
    We are a leading software development company in New York.
  </p>
  <a href="https://sparkle.test" target="_blank" class="text-blue-500"
    >Visit Website</a
  >

  <a
    href=""
    class="mt-10 bg-blue-500 hover:bg-blue-600 text-white font-bold w-full py-2
px-4 rounded-full flex items-center justify-center"
    ><i class="fas fa-bookmark mr-3"></i> Bookmark Listing</a
  >
</aside>
```

Let's replace the company logo, name, description and website with the job object data:

```
<aside class="bg-white rounded-lg shadow-md p-3">
  <h3 class="text-xl text-center mb-4 font-bold">Company Info</h3>
  <img
    src="/images/{{$job->company_logo}}"
    alt="{{$job->company_name}}"
    class="w-full rounded-lg mb-4 m-auto"
  />
  <h4 class="text-lg font-bold">{{$job->company_name}}</h4>
  <p class="text-gray-700 text-lg my-3">{{$job->company_description}}</p>
  <a href="{{$job->company_website}}" target="_blank" class="text-blue-500"
    >Visit Website</a
  >
```

```
    <a
      href=""
      class="mt-10 bg-blue-500 hover:bg-blue-600 text-white font-bold w-full py-2
px-4 rounded-full flex items-center justify-center"
      ><i class="fas fa-bookmark mr-3"></i> Bookmark Listing</a
    >
  </aside>
```

Now we have the job details page. We will come back to the edit and delete buttons and the map later. For now, we have a nice looking job details page.

5 - Create Job Page

Right now our create page looks horrible and only has a title and description field.

Let's open the `/resources/views/jobs/create.blade.php` file as well as the `_theme_files/create-job.html` file. Copy the entire `<div>` within the `<main>` tag in the theme file and paste it inside of the `<x-layout>` tags in the `/resources/views/jobs/create.blade.php` file. Forget the stuff we had there. We will re-add it where we need it.

The page should look like this at the moment:

```
<x-layout>
  <div class="bg-white mx-auto p-8 rounded-lg shadow-md w-full md:max-w-3xl">
    <h2 class="text-4xl text-center font-bold mb-4">Create Job Listing</h2>
    <form method="POST" action="/jobs" enctype="multipart/form-data">
      <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
        Job Info
      </h2>

      <div class="mb-4">
        <label class="block text-gray-700" for="title">Job Title</label>
        <input
          id="title"
          type="text"
          name="title"
          class="w-full px-4 py-2 border rounded focus:outline-none"
          placeholder="Software Engineer"
        />
      </div>

      <div class="mb-4">
        <label class="block text-gray-700" for="description"
          >Job Description</label
        >
        <textarea
          cols="30"
          rows="7"
          id="description"
          name="description"
```

```
          class="w-full px-4 py-2 border rounded focus:outline-none"
          placeholder="We are seeking a skilled and motivated Software Developer
to join our growing development team. In this role, you will be responsible for
designing, developing, and maintaining high-quality software solutions that meet
our clients' needs. You will work closely with cross-functional teams to deliver
robust applications and improve existing systems."
        ></textarea>
      </div>

      <div class="mb-4">
        <label class="block text-gray-700" for="salary">Annual Salary</label>
        <input
          id="salary"
          type="text"
          name="salary"
          class="w-full px-4 py-2 border rounded focus:outline-none"
          placeholder="90000"
        />
      </div>

      <div class="mb-4">
        <label class="block text-gray-700" for="requirements"
          >Requirements</label
        >
        <textarea
          id="requirements"
          name="requirements"
          class="w-full px-4 py-2 border rounded focus:outline-none"
          placeholder="Bachelor's degree in Computer Science"
        ></textarea>
      </div>

      <div class="mb-4">
        <label class="block text-gray-700" for="benefits">Benefits</label>
        <textarea
          id="benefits"
          name="benefits"
          class="w-full px-4 py-2 border rounded focus:outline-none"
          placeholder="Health insurance, 401k, paid time off"
        ></textarea>
      </div>

      <div class="mb-4">
        <label class="block text-gray-700" for="tags"
          >Tags (comma-separated)</label
        >
        <input
          id="tags"
          type="text"
          name="tags"
          class="w-full px-4 py-2 border rounded focus:outline-none"
          placeholder="development,coding,java,python"
        />
      </div>
```

```
        <div class="mb-4">
          <label class="block text-gray-700" for="job_type">Job Type</label>
          <select
            id="job_type"
            name="job_type"
            class="w-full px-4 py-2 border rounded focus:outline-none"
          >
            <option value="Full-Time" selected>Full-Time</option>
            <option value="Part-Time">Part-Time</option>
            <option value="Contract">Contract</option>
            <option value="Temporary">Temporary</option>
            <option value="Internship">Internship</option>
            <option value="Volunteer">Volunteer</option>
            <option value="On-Call">On-Call</option>
          </select>
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="remote">Remote</label>
          <select
            id="remote"
            name="remote"
            class="w-full px-4 py-2 border rounded focus:outline-none"
          >
            <option value="false">No</option>
            <option value="true">Yes</option>
          </select>
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="address">Address</label>
          <input
            id="address"
            type="text"
            name="address"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="123 Main St"
          />
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="city">City</label>
          <input
            id="city"
            type="text"
            name="city"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="Albany"
          />
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="state">State</label>
```

```
          <input
            id="state"
            type="text"
            name="state"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="NY"
          />
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="zipcode">ZIP Code</label>
          <input
            id="zipcode"
            type="text"
            name="zipcode"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="12201"
          />
        </div>

        <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
          Company Info
        </h2>

        <div class="mb-4">
          <label class="block text-gray-700" for="company_name"
            >Company Name</label
          >
          <input
            id="company_name"
            type="text"
            name="company_name"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="Company name"
          />
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="company_description"
            >Company Description</label
          >
          <textarea
            id="company_description"
            name="company_description"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="Company Description"
          ></textarea>
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="company_website"
            >Company Website</label
          >
          <input
```

```
            id="company_website"
            type="text"
            name="company_website"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="Enter website"
          />
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="contact_phone"
            >Contact Phone</label
          >
          <input
            id="contact_phone"
            type="text"
            name="contact_phone"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="Enter phone"
          />
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="contact_email"
            >Contact Email</label
          >
          <input
            id="contact_email"
            type="email"
            name="contact_email"
            class="w-full px-4 py-2 border rounded focus:outline-none"
            placeholder="Email where you want to receive applications"
          />
        </div>

        <div class="mb-4">
          <label class="block text-gray-700" for="company_logo"
            >Company Logo</label
          >
          <input
            id="company_logo"
            type="file"
            name="company_logo"
            class="w-full px-4 py-2 border rounded focus:outline-none"
          />
        </div>

        <button
          type="submit"
          class="w-full bg-green-500 hover:bg-green-600 text-white px-4 py-2 my-3
rounded focus:outline-none"
        >
          Save
        </button>
      </form>
```

```
  </div>
</x-layout>
```

There are a few things that we need to add here.

- @csrf - this is a security feature to prevent cross-site request forgery attacks.
- Set the action to /jobs/store
- Add the @error directive to each input field to display the error message if there is one.
- Add the `old()` function to each input field to prefill the input field with the old value if there is one.

More About @csrf

We added this directive a while ago, but I didn't really explain it and I want to explain it now.

The @csrf directive is a built-in directive that adds a hidden input field to the form with the value of the CSRF token. This is a security feature to prevent cross-site request forgery attacks. This is a type of attack where a malicious website can trick a user into performing actions on another website where the user is authenticated. The attacker can trick the user into submitting a form that performs an action on the other website. The CSRF token is a unique token that is generated for each session and is used to verify that the form submission is coming from the correct website. So this is essential for security.

Alright, here is the next version of the form:

```
<x-layout>
  <div class="bg-white mx-auto p-8 rounded-lg shadow-md w-full md:max-w-3xl">
    <h2 class="text-4xl text-center font-bold mb-4">Create Job Listing</h2>

    <!-- Form Start -->
    <form method="POST" action="{{ route('jobs.store') }}"
enctype="multipart/form-data">
      @csrf

      <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">Job Info</h2>

      <!-- Job Title -->
      <div class="mb-4">
        <label class="block text-gray-700" for="title">Job Title</label>
        <input id="title" type="text" name="title" value="{{ old('title') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('title') border-red-500 @enderror"
          placeholder="Software Engineer" />
        @error('title')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Job Description -->
      <div class="mb-4">
        <label class="block text-gray-700" for="description">Job
Description</label>
```

```
        <textarea cols="30" rows="7" id="description" name="description"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('description') border-red-500 @enderror"
          placeholder="We are seeking a skilled and motivated Software
Developer...">{{ old('description') }}</textarea>
        @error('description')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Annual Salary -->
      <div class="mb-4">
        <label class="block text-gray-700" for="salary">Annual Salary</label>
        <input id="salary" type="number" name="salary" value="{{ old('salary') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('salary') border-red-500 @enderror"
          placeholder="90000" />
        @error('salary')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Requirements -->
      <div class="mb-4">
        <label class="block text-gray-700" for="requirements">Requirements</label>
        <textarea id="requirements" name="requirements"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('requirements') border-red-500 @enderror"
          placeholder="Bachelor's degree in Computer Science">{{
old('requirements') }}</textarea>
        @error('requirements')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Benefits -->
      <div class="mb-4">
        <label class="block text-gray-700" for="benefits">Benefits</label>
        <textarea id="benefits" name="benefits"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('benefits') border-red-500 @enderror"
          placeholder="Health insurance, 401k, paid time off">{{ old('benefits')
}}</textarea>
        @error('benefits')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Tags -->
      <div class="mb-4">
        <label class="block text-gray-700" for="tags">Tags (comma-separated)
</label>
        <input id="tags" type="text" name="tags" value="{{ old('tags') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none @error('tags')
```

```
border-red-500 @enderror"
          placeholder="development,coding,java,python" />
        @error('tags')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Job Type -->
      <div class="mb-4">
        <label class="block text-gray-700" for="job_type">Job Type</label>
        <select id="job_type" name="job_type"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('job_type') border-red-500 @enderror">
          <option value="Full-Time" {{ old('job_type') == 'Full-Time' ? 'selected'
: '' }}>Full-Time</option>
          <option value="Part-Time" {{ old('job_type') == 'Part-Time' ? 'selected'
: '' }}>Part-Time</option>
          <option value="Contract" {{ old('job_type') == 'Contract' ? 'selected' :
'' }}>Contract</option>
          <option value="Temporary" {{ old('job_type') == 'Temporary' ? 'selected'
: '' }}>Temporary</option>
          <option value="Internship" {{ old('job_type') == 'Internship' ?
'selected' : '' }}>Internship</option>
          <option value="Volunteer" {{ old('job_type') == 'Volunteer' ? 'selected'
: '' }}>Volunteer</option>
          <option value="On-Call" {{ old('job_type') == 'On-Call' ? 'selected' :
'' }}>On-Call</option>
        </select>
        @error('job_type')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Remote -->
      <div class="mb-4">
        <label class="block text-gray-700" for="remote">Remote</label>
        <select id="remote" name="remote"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('remote') border-red-500 @enderror">
          <option value="0" {{ old('remote')==false ? 'selected' : ''
}}>No</option>
          <option value="1" {{ old('remote')==true ? 'selected' : ''
}}>Yes</option>
        </select>
        @error('remote')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">Company
Info</h2>

      <!-- Address -->
      <div class="mb-4">
```

```
        <label class="block text-gray-700" for="address">Address</label>
        <input id="address" type="text" name="address" value="{{ old('address')
}}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('address') border-red-500 @enderror"
          placeholder="123 Main St" />
        @error('address')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- City -->
      <div class="mb-4">
        <label class="block text-gray-700" for="city">City</label>
        <input id="city" type="text" name="city" value="{{ old('city') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none @error('city')
border-red-500 @enderror"
          placeholder="Albany" />
        @error('city')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- State -->
      <div class="mb-4">
        <label class="block text-gray-700" for="state">State</label>
        <input id="state" type="text" name="state" value="{{ old('state') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('state') border-red-500 @enderror"
          placeholder="NY" />
        @error('state')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- ZIP Code -->
      <div class="mb-4">
        <label class="block text-gray-700" for="zipcode">ZIP Code</label>
        <input id="zipcode" type="text" name="zipcode" value="{{ old('zipcode')
}}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('zipcode') border-red-500 @enderror"
          placeholder="12201" />
        @error('zipcode')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Company Name -->
      <div class="mb-4">
        <label class="block text-gray-700" for="company_name">Company Name</label>
        <input id="company_name" type="text" name="company_name" value="{{
old('company_name') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none
```

```
@error('company_name') border-red-500 @enderror"
          placeholder="Company name" />
        @error('company_name')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Company Description -->
      <div class="mb-4">
        <label class="block text-gray-700" for="company_description">Company
Description</label>
        <textarea id="company_description" name="company_description"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('company_description') border-red-500 @enderror"
          placeholder="Company Description">{{ old('company_description') }}
</textarea>
        @error('company_description')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Company Website -->
      <div class="mb-4">
        <label class="block text-gray-700" for="company_website">Company
Website</label>
        <input id="company_website" type="url" name="company_website" value="{{
old('company_website') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('company_website') border-red-500 @enderror"
          placeholder="Enter website" />
        @error('company_website')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Contact Phone -->
      <div class="mb-4">
        <label class="block text-gray-700" for="contact_phone">Contact
Phone</label>
        <input id="contact_phone" type="text" name="contact_phone" value="{{
old('contact_phone') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('contact_phone') border-red-500 @enderror"
          placeholder="Enter phone" />
        @error('contact_phone')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Contact Email -->
      <div class="mb-4">
        <label class="block text-gray-700" for="contact_email">Contact
Email</label>
        <input id="contact_email" type="email" name="contact_email" value="{{
```

```
old('contact_email') }}"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('contact_email') border-red-500 @enderror"
          placeholder="Email where you want to receive applications" />
        @error('contact_email')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Company Logo -->
      <div class="mb-4">
        <label class="block text-gray-700" for="company_logo">Company Logo</label>
        <input id="company_logo" type="file" name="company_logo"
          class="w-full px-4 py-2 border rounded focus:outline-none
@error('company_logo') border-red-500 @enderror" />
        @error('company_logo')
          <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
        @enderror
      </div>

      <!-- Submit Button -->
      <button type="submit"
        class="w-full bg-green-500 hover:bg-green-600 text-white px-4 py-2 my-3
rounded focus:outline-none">
        Save
      </button>
    </form>
    <!-- Form End -->

  </div>
</x-layout>
```

If you submit an empty form, you will see the errors for the title and description because that is all we have handled in the controller method so far. We will add the rest of the fields soon, but I would like to have some components for our form inputs that include the labels and error messages. We will do that in the next lesson.

6 - Form Input Components

So we have our form, but the code is very repetitive. We have a lot of input fields and labels and error messages. We can create a component for inputs like text, textarea, select, etc. This is a good practice because it makes our code more readable and easier to maintain. If you have ever used a frontend framework like React, this should look familiar.

Create a new folder in the components folder called `inputs`.

Text Input Component

Let's generate a new component called `Text`:

```
php artisan make:component Text
```

This will create a file at `app/View/Text.php` and `resources/views/text.blade.php`.

Move the blade file to the `resources/views/components/inputs` folder.

You have to change the view path in the `app/View/Text.php` file:

```
public function render()
{
    return view('components.inputs.text'); // updated path
}
```

Now add the following to the blade component:

```
@props(['id', 'name', 'label' => null, 'type' => 'text', 'value' => '',
'placeholder' => ''])

<div class="mb-4">
  @if($label)
  <label class="block text-gray-700" for="{{ $id }}">{{ $label }}</label>
  @endif
  <input
    id="{{ $id }}"
    type="{{ $type }}"
    name="{{ $name }}"
    value="{{ old($name, $value) }}"
    class="w-full px-4 py-2 border rounded focus:outline-none @error($name)
border-red-500 @enderror"
    placeholder="{{ $placeholder }}"
  />
  @error($name)
  <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
  @enderror
</div>
```

This is exactly what we have in the form but it is all dynamic. We can use this component for all of our text inputs. Before we do that though, let's create components for the rest of our input types.

Textarea Component

Let's generate a new component called `TextArea`:

```
php artisan make:component TextArea
```

This will create a file at `app/View/TextArea.php` and `resources/views/text-area.blade.php`.

Move the blade file to the `resources/views/components/inputs` folder.

You have to change the view path in the `app/View/TextArea.php` file:

```
public function render()
{
    return view('components.inputs.textarea'); // updated path
}
```

Now add the following to the blade component:

```
@props(['id', 'name', 'label' => null, 'value' => '', 'placeholder' => ''])

<div class="mb-4">
  @if($label)
  <label class="block text-gray-700" for="{{ $id }}">{{ $label }}</label>
  @endif
  <textarea
    id="{{ $id }}"
    name="{{ $name }}"
    cols="30"
    rows="7"
    class="w-full px-4 py-2 border rounded focus:outline-none @error($name)
border-red-500 @enderror"
    placeholder="{{ $placeholder }}"
  >
{{ old($name, $value) }}</textarea
  >
  @error($name)
  <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
  @enderror
</div>
```

Select Component

Let's generate a new component called `select`:

```
php artsian make:component select
```

This will create a file at `app/View/select.php` and `resources/views/select.blade.php`.

Move the blade file to the `resources/views/components/inputs` folder.

You have to change the view path in the `app/View/select.php` file:

```
public function render()
{
    return view('components.inputs.select'); // updated path
}
```

Now add the following to the blade component:

```
@props(['id', 'name', 'label' => null, 'options' => [], 'value' => ''])

<div class="mb-4">
  @if($label)
  <label class="block text-gray-700" for="{{ $id }}">{{ $label }}</label>
  @endif
  <select id="{{ $id }}" name="{{ $name }}"
    class="w-full px-4 py-2 border rounded focus:outline-none @error($name)
border-red-500 @enderror">
    @foreach($options as $optionValue => $optionLabel)
    <option value="{{ $optionValue }}" {{ old($name, $value)==$optionValue ?
'selected' : '' }}>
      {{ $optionLabel }}
    </option>
    @endforeach
  </select>
  @error($name)
  <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
  @enderror
</div>
```

This will take in an array of options and display them as a select input.

File Component

Let's generate a new component called `file`:

```
php artsian make:component file
```

This will create a file at `app/View/file.php` and `resources/views/file.blade.php`.

Move the blade file to the `resources/views/components/inputs` folder.

You have to change the view path in the `app/View/file.php` file:

```
public function render()
{
    return view('components.inputs.file'); // updated path
}
```

Now add the following to the blade component:

```
@props(['id', 'name', 'label' => null])

<div class="mb-4">
  @if($label)
  <label class="block text-gray-700" for="{{ $id }}">{{ $label }}</label>
  @endif
  <input
    id="{{ $id }}"
    type="file"
    name="{{ $name }}"
    class="w-full px-4 py-2 border rounded focus:outline-none @error($name)
border-red-500 @enderror"
  />
  @error($name)
  <p class="text-red-500 text-sm mt-1">{{ $message }}</p>
  @enderror
</div>
```

Here is the final form with the new components:

```
<x-layout>
  <div class="bg-white mx-auto p-8 rounded-lg shadow-md w-full md:max-w-3xl">
    <h2 class="text-4xl text-center font-bold mb-4">Create Job Listing</h2>

    <!-- Form Start -->
    <form
      method="POST"
      action="{{ route('jobs.store') }}"
      enctype="multipart/form-data"
    >
      @csrf

      <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
        Job Info
      </h2>

      <!-- Job Title -->
      <x-inputs.text
        id="title"
        name="title"
        label="Job Title"
        placeholder="Software Engineer"
      />

      <x-inputs.textarea
        id="description"
        name="description"
```

```
          label="Job Description"
          placeholder="We are seeking a skilled and motivated Software Developer..."
        />

        <x-inputs.text
          id="salary"
          name="salary"
          label="Annual Salary"
          type="number"
          placeholder="90000"
        />

        <x-inputs.textarea
          id="requirements"
          name="requirements"
          label="Requirements"
          placeholder="Bachelor's degree in Computer Science"
        />

        <x-inputs.textarea
          id="benefits"
          name="benefits"
          label="Benefits"
          placeholder="Health insurance, 401k, paid time off"
        />

        <x-inputs.text
          id="tags"
          name="tags"
          label="Tags (comma-separated)"
          type="text"
          placeholder="development,coding,java,python"
        />

        <x-inputs.select
          id="job_type"
          name="job_type"
          label="Job Type"
          :options="['Full-Time' => 'Full-Time', 'Part-Time' => 'Part-Time',
'Contract' => 'Contract', 'Temporary' => 'Temporary', 'Internship' =>
'Internship', 'Volunteer' => 'Volunteer', 'On-Call' => 'On-Call']"
          value="{{ old('job_type') }}"
        />

        <x-inputs.select
          id="remote"
          name="remote"
          label="Remote"
          :options="[0 => 'No', 1 => 'Yes']"
        />

        <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
          Company Info
        </h2>
```

```
        <x-inputs.text
          id="address"
          name="address"
          label="Address"
          type="text"
          placeholder="123 Main St"
        />

        <x-inputs.text
          id="city"
          name="city"
          label="City"
          type="text"
          placeholder="Albany"
        />

        <x-inputs.text
          id="state"
          name="state"
          label="State"
          type="text"
          placeholder="NY"
        />

        <x-inputs.text
          id="zipcode"
          name="zipcode"
          label="ZIP Code"
          type="text"
          placeholder="12201"
        />

        <x-inputs.text
          id="company_name"
          name="company_name"
          label="Company Name"
          type="text"
          placeholder="Company name"
        />

        <x-inputs.textarea
          id="company_description"
          name="company_description"
          label="Company Description"
          placeholder="Company Description"
        />

        <x-inputs.text
          id="company_website"
          name="company_website"
          label="Company Website"
          type="url"
          placeholder="Enter website"
```

```
        />

        <x-inputs.text id="contact_phone" name="contact_phone" label="Contact
        Phone" type="text" " placeholder=" Enter phone" />

        <x-inputs.text
          id="contact_email"
          name="contact_email"
          label="Contact Email"
          type="email"
          placeholder="Email where you want to receive applications"
        />

        <x-inputs.file
          id="company_logo"
          name="company_logo"
          label="Company Logo"
        />

        <!-- Submit Button -->
        <button
          type="submit"
          class="w-full bg-green-500 hover:bg-green-600 text-white px-4 py-2 my-3
rounded focus:outline-none"
        >
          Save
        </button>
      </form>
    </div>
</x-layout>
```

7 - Finish Form Validation

We have our form submitting data to the store method in the `JobController`. We need to add the fields to the store method for validation and submission.

Let's open the `store` method in the `App\Http\Controllers\JobController` class. Right now it looks like this:

```
 public function store(Request $request): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
    ]);

    // Create a new job listing with the validated data
    Job::create([
        'title' => $validatedData['title'],
```

```
            'description' => $validatedData['description'],
        ]);

        return redirect()->route('jobs.index');
    }
```

This is ok if we only want a title and a description, but we have tons more fields now, so we need to handle those. Let's update the `store` method to handle all of the new fields. Here is the updated `store` method:

```
    public function store(Request $request): RedirectResponse
    {
        // Validate the incoming request data
        $validatedData = $request->validate([
            'title' => 'required|string|max:255',
            'description' => 'required|string',
            'salary' => 'required|integer',
            'tags' => 'nullable|string',
            'job_type' => 'required|string',
            'remote' => 'required|boolean',
            'requirements' => 'nullable|string',
            'benefits' => 'nullable|string',
            'address' => 'nullable|string',
            'city' => 'required|string',
            'state' => 'required|string',
            'zipcode' => 'required|string',
            'contact_email' => 'required|email',
            'contact_phone' => 'nullable|string',
            'company_name' => 'required|string',
            'company_description' => 'nullable|string',
            'company_logo' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
            'company_website' => 'nullable|url',
        ]);

        // Create a new job listing with the validated data
        Job::create($validatedData);

        return redirect()->route('jobs.index');
    }
```

Handle User ID

Another issue is that ultimately, all listings will be connected to a user ID but we do not have authentication yet. So for now, let's hard code the user ID to 1.

Add this line under the `$validatedData` variable:

```
    // Add the hardcoded user_id
      $validatedData['user_id'] = 1;
```

Send Success Message

When we redirect, we can attach a success message to the session:

```
    return redirect()->route('jobs.index')->with('success', 'Job listing created
successfully!');
```

We will have to handle this in the view. We will do that in the next lesson.

Now if you try and submit an empty form, you will get all kinds of errors. If you fill out the form correctly, you will be redirected to the jobs index page. From this point on, I would suggest using a browser extension such as Fake Filler to fill out the form. Otherwise, you will need to manually fill out the form and it's a long form.

We are now able to store all of the new fields in the database.

8 - Alert Components

In this lesson, I want to handle the success or error messages in the session when we redirect with an alert component.

Make sure in the `JobController` you have the following in the `store` method:

```
return redirect()->route('jobs.index')->with('success', 'Job listing created
successfully!');
```

Create a new component called `Alert`. Run the following command:

```
php artisan make:component Alert
```

Open the `resources/views/components/Alert.blade.php` file and add the following code:

```
@props(['type', 'message'])

@if(session()->has('success') || session()->has('error'))
<div class="p-4 mb-4 text-sm text-white {{ $type === 'success' ? 'bg-green-500' :
'bg-red-500' }} rounded">
    {{ $message }}
</div>
@endif
```

Let's add the component to the layout. Open the `resources/views/components/layout.blade.php` file and add the following code above the `{{ $slot }}`:

```
<!-- Display alert messages -->
@if (session('success'))
<x-alert type="success" message="{{ session('success') }}" />
@endif

@if (session('error'))
<x-alert type="error" message="{{ session('error') }}" />
@endif
```

Now submit a new job listing and you should see the success message.

One issue we have that I don't like is that the message stays there until we refresh the page. We can change that with a little JavaScript. For interactive stuff like this in Laravel, I like to use Alpine.js. We will do that in the next lesson.

9 - Dismiss Alert & Alpine JS

We are going to make it so we the alert goes away after a few seconds. Since this is something that is interactive and happens on the client-side, we need to use JavaScript. You can use Vanilla JS if you want by adding it to the `public/js/script.js` file, however I prefer to use Alpine JS for this because it is cleaner and we don't even need to write any JavaScript. We do it all from the Blade file. Alpine is a library that gives us a bunch of attributes and directives that we can use in our HTML and it is often used with Laravel to make things more interactive.

Alpine CDN

There are a few ways to use Alpine JS. We can install it via npm or yarn, but we can also use a CDN. We are going to use the CDN. This makes things much easier. Open the `resources/views/components/layout.blade.php` file and add the following code to the `<head>` tag:

```
<script
  defer
  src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"
></script>
```

Alpine Directives

Now open the alert component at `resources/views/components/Alert.blade.php` and add the following code:

```
@props(['type', 'message'])

<div
  x-data="{ show: true }"
  x-init="setTimeout(() => show = false, 5000)"
```

```
    x-show="show"
    class="p-4 mb-4 text-sm text-white {{ $type === 'success' ? 'bg-green-500' :
'bg-red-500' }} rounded">
      {{ session($type) }}
</div>
```

We are using the `x-data` directive to create a new object with a `show` property. We are setting it to `true` by default. We are using the `x-init` directive to set a timeout of 5 seconds to set the `show` property to `false`. We are using the `x-show` directive to show the alert if `show` is `true`.

Just to show you what we would need to do if we were using Vanilla JS, we can add the following code to the `public/js/script.js` file:

```
document.querySelectorAll('[id$="-alert"]').forEach(function (alert) {
  // Get the duration from the data attribute
  const duration = parseInt(alert.getAttribute('data-duration'), 10);

  // Set a timeout to remove the alert after the specified duration
  setTimeout(function () {
    alert.style.opacity = 0;
    setTimeout(function () {
      alert.remove();
    }, 600); // Match this duration with your CSS transition time
  }, duration);
});
```

You can see why Alpine JS is so much easier to use.

Now your alert should go away after a few seconds.

Using Alpine JS For The Mobile Menu

We have our hamburger menu working with vanilla JS, but we can use Alpine JS to make it even easier. Open the `resources/views/components/header.blade.php` file and make some changes.

First, we need to add an "open" state. This has to be on a parent of the button and menu. So let's add it to the `<header>` element and set it to false:

```
<header class="bg-blue-900 text-white p-4" x-data="{ open: false }">...</header>
```

Next, let's have the hamburger menu button toggle the open state. We can do this by adding the following code to the button:

```
<button @click="open = !open" class="text-white md:hidden flex items-center">
  <i class="fa fa-bars text-2xl"></i>
</button>
```

I removed the id of `hamburger`. We don't need it anymore. We are using the `@click` directive to toggle the `open` property. If it is `true`, it will be set to `false` and vice versa.

Finally, we need to show the menu if `open` is `true`. We can do this by adding the following code to the `<nav>` element:

```
<nav
  x-show="open"
  @click.away="open = false"
  class="md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"
></nav>
```

I removed the class of `hidden`. We don't need it anymore. We are using the `x-show` directive to show the menu if `open` is `true`. We are using the `@click.away` directive to close the menu if the user clicks away from it.

Now you can delete the Javascript from the `public/js/script.js` file. We don't need it anymore. The mobile menu should still work the same way.

10 - Handle Optional Data

We have some fields that are optional or nullable. I want to handle displaying these fields in our layouts.

Company On Card

The logo is not required. In fact right now we don't even have the ability to upload an image. The card however is trying to render the image. Let's open the `/resources/views/components/job-card.blade.php` file and wrap the logo in an `if` statement.

```
@if($job->company_logo)
<img
  src="/images/{{ $job->company_logo }}"
  alt="{{$job->company_name}}"
  class="w-14"
/>
@endif
```

Job Details Page

There are a lot of optional fields on the job details/show page that I only want to display if they exist.

Here are the optional fields:

- requirements
- benefits

- tags
- company_logo
- company_website
- company_phone
- company_description

Now let's open the job details page at `/resources/views/jobs/show.blade.php` and add some logic to handle the optional fields.

Tags

```
@if ($job->tags)
<li class="mb-2">
  <strong>Tags:</strong>
  {{ ucwords(str_replace(',', ', ', $job->tags)) }}
</li>
@endif
```

Benefits & Requirements

```
@if ($job->requirements || $job->benefits)
<h2 class="text-xl font-semibold mb-4">Job Details</h2>
<div class="rounded-lg shadow-md bg-white p-4">
  @if ($job->requirements)
  <h3 class="text-lg font-semibold mb-2 text-blue-500">Job Requirements</h3>
  <p>{{$job->requirements}}</p>
  @endif @if ($job->benefits)
  <h3 class="text-lg font-semibold mt-4 mb-2 text-blue-500">Benefits</h3>
  <p>{{$job->benefits}}</p>
  @endif
</div>
@endif
```

Company Fields

```
@if ($job->company_logo)
<img
  src="/images/{{$job->company_logo}}"
  alt="{{$job->company_name}}"
  class="w-full rounded-lg mb-4 m-auto"
/>
@endif @if ($job->company_name)
<h4 class="text-lg font-bold">{{$job->company_name}}</h4>
@endif @if ($job->company_description)
<p class="text-gray-700 text-lg my-3">{{$job->company_description}}</p>
@endif @if ($job->company_website)
```

```
<a href="{{$job->company_website}}" target="_blank" class="text-blue-500"
  >Visit Website</a
>
@endif @if ($job->company_phone)
<p class="text-gray-700 text-lg mt-2">Phone: {{$job->company_phone}}</p>
@endif
```

Here is the final code for now:

```
<x-layout>
  <div class="grid grid-cols-1 md:grid-cols-4 gap-6">
    <section class="md:col-span-3">
      <div class="rounded-lg shadow-md bg-white p-3">
        <div class="flex justify-between items-center">
          <a class="block p-4 text-blue-700" href="{{ route('jobs.index') }}">
            <i class="fa fa-arrow-alt-circle-left"></i>
            Back To Listings
          </a>
          <div class="flex space-x-3 ml-4">
            <a
              href="/edit"
              class="px-4 py-2 bg-blue-500 hover:bg-blue-600 text-white rounded"
              >Edit</a
            >
            <!-- Delete Form -->
            <form method="POST">
              <button
                type="submit"
                class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
              >
                Delete
              </button>
            </form>
            <!-- End Delete Form -->
          </div>
        </div>
        <div class="p-4">
          <h2 class="text-xl font-semibold">{{$job->title}}</h2>
          <p class="text-gray-700 text-lg mt-2">{{$job->description}}</p>
          <ul class="my-4 bg-gray-100 p-4">
            <li class="mb-2"><strong>Job Type:</strong> {{$job->job_type}}</li>
            <li class="mb-2">
              <strong>Remote:</strong> {{$job->remote ? 'Yes' : 'No'}}
            </li>
            <li class="mb-2">
              <strong>Salary:</strong> ${{ number_format($job->salary) }}
            </li>
            <li class="mb-2">
              <strong>Site Location:</strong> {{$job->city}}, {{$job->state}}
            </li>
            @if ($job->tags)
            <li class="mb-2">
```

```
              <strong>Tags:</strong>
              {{ ucwords(str_replace(',', ', ', $job->tags)) }}
            </li>
            @endif
          </ul>
        </div>
      </div>

      <div class="container mx-auto p-4">
        @if ($job->requirements || $job->benefits)
        <h2 class="text-xl font-semibold mb-4">Job Details</h2>
        <div class="rounded-lg shadow-md bg-white p-4">
          @if ($job->requirements)
          <h3 class="text-lg font-semibold mb-2 text-blue-500">
            Job Requirements
          </h3>
          <p>{{$job->requirements}}</p>
          @endif @if ($job->benefits)
          <h3 class="text-lg font-semibold mt-4 mb-2 text-blue-500">
            Benefits
          </h3>
          <p>{{$job->benefits}}</p>
          @endif
        </div>
        @endif
        <p class="my-5">
          Put "Job Application" as the subject of your email and attach your
          resume.
        </p>
        <a
          href="mailto:{{$job->contact_email}}"
          class="block w-full text-center px-5 py-2.5 shadow-sm rounded border
text-base font-medium cursor-pointer text-indigo-700 bg-indigo-100 hover:bg-
indigo-200"
        >
          Apply Now
        </a>
      </div>

      <div class="bg-white p-6 rounded-lg shadow-md mt-6">
        <div id="map"></div>
      </div>
    </section>

    <aside class="bg-white rounded-lg shadow-md p-3">
      <h3 class="text-xl text-center mb-4 font-bold">Company Info</h3>
      @if ($job->company_logo)
      <img
        src="/images/{{$job->company_logo}}"
        alt="{{$job->company_name}}"
        class="w-full rounded-lg mb-4 m-auto"
      />
      @endif @if ($job->company_name)
      <h4 class="text-lg font-bold">{{$job->company_name}}</h4>
```

```
        @endif @if ($job->company_description)
        <p class="text-gray-700 text-lg my-3">{{$job->company_description}}</p>
        @endif @if ($job->company_website)
        <a href="{{$job->company_website}}" target="_blank" class="text-blue-500"
          >Visit Website</a
        >
        @endif @if ($job->company_phone)
        <p class="text-gray-700 text-lg mt-2">Phone: {{$job->company_phone}}</p>
        @endif

        <a
          href=""
          class="mt-10 bg-blue-500 hover:bg-blue-600 text-white font-bold w-full py-
2 px-4 rounded-full flex items-center justify-center"
          ><i class="fas fa-bookmark mr-3"></i> Bookmark Listing</a
        >
      </aside>
    </div>
</x-layout>
```

11 - Uploading Files

We are able to create a new job listing, but we are missing the company logo upload functionality. Let's do that now.

We already have the table column for the image path in our database and we also have the image upload field in the form. We just need to add the logic to handle the file upload.

Let's open the `JobController` and add this right above where we create the job:

```
dd($request->file('company_logo'));
```

This will print out the file object and stop the script. Now, go to the job creation form and create a new job listing. You should see the file object printed out. This includes things like the file name, the file extension, the file mime type, file path, etc.

There is a method called `store()` that we can use on this object. This method will store the file in the `storage/app/public` directory. We can also specify a directory within that public directory.

Create a Symlink

Before we can access the file from the browser, we need to create a symlink. Run the following command:

```
php artisan storage:link
```

This will create a symlink from the `public/storage` directory to the `storage/app/public` directory.

Now remove the `dd` and add this to the `store()` method:

```
 // Check if a file was uploaded
if ($request->hasFile('company_logo')) {
    // Store the file and get the path
    $path = $request->file('company_logo')->store('logos', 'public');

    // Add the path to the validated data array
    $validatedData['company_logo'] = $path;
}
```

Now try and upload a listing with a file.

You will not see the image in the website yet, but you should see the image in the `storage/app/public/logos` directory.

The URL to the image would be:

```
http://127.0.0.1:8000/storage/logos/YOURIMAGE.png
```

Update the Card Component & Details Page Image Path

Now that we have the image path, we need to update the card component and the job details page to display the image.

Open the `/resources/views/components/job-card.blade.php` file and change the image path to the following:

```
<img
  src="/storage/{{ $job->company_logo }}"
  alt="{{$job->company_name}}"
  class="w-14"
/>
```

Now open the `/resources/views/job-details.blade.php` file and change the image path to the following:

```
<img
  src="/storage/{{$job->company_logo}}"
  alt="{{$job->company_name}}"
  class="w-full rounded-lg mb-4 m-auto"
/>
```

Now you will not see any of the other images. However, if you want to keep the sample images, just move them to the `storage/app/public/logos` directory. Now you will see your sample data images and your newly uploaded images.

12 - Edit Page

We have the "C" and the "R" in CRUD. Now we need to add the "U" for update. We need to create an edit page where we can update the job listing. We already have our methods in place and if you have been following along, you should have the edit button on the details page and it should take you to `http::localhost:8000/jobs/:id/edit`. I know it may be a bit confusing because a lot of other frameworks use the /job/edit/:id but we are using the /jobs/:id/edit. This is just a convention that Laravel uses.

Controller Method

We need to fetch the job listing that we want to edit and pass it to the view, which we'll create in a minute.

We can either use the `find` method to find the job listing by the id.

```
 public function edit(string $id): View
{
    // Fetch the current job listing
    $job = Job::find($id);
    return view('jobs.edit')->with('job', $job);
}
```

Or, we can just use model binding and pass the job object to the method.

```
 public function edit(Job $job): View
{
    return view('jobs.edit')->with('job', $job);
}
```

I will use the latter.

Let's create the edit view. Create a new file in the `resources/views/jobs` directory called `edit.blade.php`. We can copy the create page and make the changes. Here are the changes we need to make.

Heading

Change the heading to "Edit Job Listing":

```
<h2 class="text-4xl text-center font-bold mb-4">Edit Job Listing</h2>
```

Action & `@method` Directive

We need to change the form action to point to the update route. We can use the `route` helper function to generate the URL. We need to pass in the route name and the job listing id. Here is the updated form tag:

```
<form
  action="{{ route('jobs.update', $job->id) }}"
  method="POST"
  enctype="multipart/form-data"
></form>
```

We also need to add the `@method` directive to tell Laravel that we are using the `PUT` method. Regular HTML forms can only use the `GET` and `POST` methods. We can use the `@method` directive to tell Laravel that we are using the `PUT` method. Here is the updated form tag:

```
<form
  method="POST"
  action="{{ route('jobs.update', $job->id) }}"
  enctype="multipart/form-data"
>
  @csrf
  <!-- Add this line -->
  @method('PUT')
</form>
```

Values

For the values, we want the current job listing values. We can just pass the value into each input component. Here is the final code for the edit form:

```
<x-layout>
  <div class="bg-white mx-auto p-8 rounded-lg shadow-md w-full md:max-w-3xl">
    <h2 class="text-4xl text-center font-bold mb-4">Edit Job Listing</h2>

    <!-- Form Start -->
    <form
      method="POST"
      action="{{ route('jobs.update', $job->id) }}"
      enctype="multipart/form-data"
    >
      @csrf @method('PUT')

      <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
        Job Info
      </h2>

      <!-- Job Title -->
      <x-inputs.text
        id="title"
```

```
            name="title"
            label="Job Title"
            placeholder="Software Engineer"
            :value="old('title', $job->title)"
        />

        <x-inputs.textarea
            id="description"
            name="description"
            label="Job Description"
            placeholder="We are seeking a skilled and motivated Software Developer..."
            :value="old('description', $job->description)"
        />

        <x-inputs.text
            id="salary"
            name="salary"
            label="Annual Salary"
            type="number"
            placeholder="90000"
            :value="old('salary', $job->salary)"
        />

        <x-inputs.textarea
            id="requirements"
            name="requirements"
            label="Requirements"
            placeholder="Bachelor's degree in Computer Science"
            :value="old('requirements', $job->requirements)"
        />

        <x-inputs.textarea
            id="benefits"
            name="benefits"
            label="Benefits"
            placeholder="Health insurance, 401k, paid time off"
            :value="old('benefits', $job->benefits)"
        />

        <x-inputs.text
            id="tags"
            name="tags"
            label="Tags (comma-separated)"
            placeholder="development,coding,java,python"
            :value="old('tags', $job->tags)"
        />

        <x-inputs.select
            id="job_type"
            name="job_type"
            label="Job Type"
            :options="['Full-Time' => 'Full-Time', 'Part-Time' => 'Part-Time',
'Contract' => 'Contract', 'Temporary' => 'Temporary', 'Internship' =>
'Internship', 'Volunteer' => 'Volunteer', 'On-Call' => 'On-Call']"
```

```
            :value="old('job_type', $job->job_type)"
        />

        <x-inputs.select
            id="remote"
            name="remote"
            label="Remote"
            :options="[0 => 'No', 1 => 'Yes']"
            :value="old('remote', $job->remote)"
        />

        <h2 class="text-2xl font-bold mb-6 text-center text-gray-500">
            Company Info
        </h2>

        <x-inputs.text
            id="address"
            name="address"
            label="Address"
            placeholder="123 Main St"
            :value="old('address', $job->address)"
        />

        <x-inputs.text
            id="city"
            name="city"
            label="City"
            placeholder="Albany"
            :value="old('city', $job->city)"
        />

        <x-inputs.text
            id="state"
            name="state"
            label="State"
            placeholder="NY"
            :value="old('state', $job->state)"
        />

        <x-inputs.text
            id="zipcode"
            name="zipcode"
            label="ZIP Code"
            placeholder="12201"
            :value="old('zipcode', $job->zipcode)"
        />

        <x-inputs.text
            id="company_name"
            name="company_name"
            label="Company Name"
            placeholder="Company name"
            :value="old('company_name', $job->company_name)"
        />
```

```
      <x-inputs.textarea
        id="company_description"
        name="company_description"
        label="Company Description"
        placeholder="Company Description"
        :value="old('company_description', $job->company_description)"
      />

      <x-inputs.text
        id="company_website"
        name="company_website"
        label="Company Website"
        type="url"
        placeholder="Enter website"
        :value="old('company_website', $job->company_website)"
      />

      <x-inputs.text
        id="contact_phone"
        name="contact_phone"
        label="Contact Phone"
        placeholder="Enter phone"
        :value="old('contact_phone', $job->contact_phone)"
      />

      <x-inputs.text
        id="contact_email"
        name="contact_email"
        label="Contact Email"
        type="email"
        placeholder="Email where you want to receive applications"
        :value="old('contact_email', $job->contact_email)"
      />

      <x-inputs.file
        id="company_logo"
        name="company_logo"
        label="Company Logo"
      />

      <!-- Submit Button -->
      <button
        type="submit"
        class="w-full bg-green-500 hover:bg-green-600 text-white px-4 py-2 my-3
rounded focus:outline-none"
      >
        Save
      </button>
    </form>
  </div>
</x-layout>
```

Update Method

Now we handle the actual update. This will be very similar to the `store` method. There are some changes though. First off, let's use model binding. Instead of passing in the `id` as a parameter, we can just pass in the model itself along with the request object.

```
 public function update(Request $request, Job $job){}
```

We also want to delete the image before we update it if a new logo was submitted. To do this, we need to use the `Storage` facade. We need to import the `Storage` facade at the top of the file:

```
use Illuminate\Support\Facades\Storage;
```

```
 // Check if a file was uploaded
if ($request->hasFile('company_logo')) {
    // Delete the old company logo from storage
    if ($job->company_logo) {
        Storage::delete('public/logos/' . basename($job->company_logo));
    }
    // Store the file and get the path
    $path = $request->file('company_logo')->store('logos', 'public');

    // Add the path to the validated data array
    $validatedData['company_logo'] = $path;
}
```

Finally, we want to call the `update` method on the model. We can use the `$job` variable:

```
// Update with the validated data
$job->update($validatedData);
```

Here is the entire method:

```
public function update(Request $request,  Job $job): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
        'salary' => 'required|integer',
        'tags' => 'nullable|string',
        'job_type' => 'required|string',
        'remote' => 'required|boolean',
```

```
            'requirements' => 'nullable|string',
            'benefits' => 'nullable|string',
            'address' => 'nullable|string',
            'city' => 'required|string',
            'state' => 'required|string',
            'zipcode' => 'required|string',
            'contact_email' => 'required|email',
            'contact_phone' => 'nullable|string',
            'company_name' => 'required|string|max:255',
            'company_description' => 'nullable|string',
            'company_logo' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
            'company_website' => 'nullable|url',
        ]);


        // Check if a file was uploaded
        if ($request->hasFile('company_logo')) {
            // Delete the old company logo from storage
            if ($job->company_logo) {
                Storage::delete('public/logos/' . basename($job->company_logo));
            }
            // Store the file and get the path
            $path = $request->file('company_logo')->store('logos', 'public');

            // Add the path to the validated data array
            $validatedData['company_logo'] = $path;
        }

        // Update with the validated data
        $job->update($validatedData);

        return redirect()->route('jobs.index')->with('success', 'Job listing updated
successfully!');
    }
```

I also removed the following line:

```
$validatedData['user_id'] = 1;
```

We will keep the user ID as as what is stored in the database.

Now try and edit a job listing. Try without changing the logo first and then try changing the logo. You should see the logo change on the details page and the old one should get deleted. You should also see the success message.

In the next lesson, we will add the delete functionality.

13 - Delete a Listing

We now have the "CRU" of the "CRUD" in place. Now we need to add the delete functionality.

Open the show view at `resources/views/jobs/show.blade.php`. We already have a delete button, but it's not doing anything. It is actually in it's own little form because we can not just use a link. We need to send a `DELETE` request to the server. Update that form to the following:

```
<form
  method="POST"
  action="{{ route('jobs.destroy', $job->id) }}"
  onsubmit="return confirm('Are you sure you want to delete this job?');"
>
  @csrf @method('DELETE')
  <button
    type="submit"
    class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
  >
    Delete
  </button>
</form>
```

This form is sending a `DELETE` request to the `jobs.destroy` route. I also added a confirmation. The `@method('DELETE')` directive is a hidden input field that tells Laravel to treat this request as a `DELETE` request. The `@csrf` directive is a hidden input field that Laravel uses to protect against cross-site request forgery (CSRF) attacks.

`destroy` Method

Now we need to add the `destroy` method to the `JobController`. Open the `JobController` and add the following method:

```
public function destroy(Job $job): RedirectResponse
{
    // If there is a company logo, delete it from storage
    if ($job->company_logo) {
        Storage::delete('public/logos/' . $job->company_logo);
    }

    // Delete the job
    $job->delete();

    return redirect()->route('jobs.index')->with('success', 'Job listing deleted
successfully!');
}
```

We are checking for a logo and if there is one, we delete that first. Then we delete the job and redirect with a success message.

Try deleting a job and you should see the logo deleted from storage and the job deleted from the database.

Now we have full CRUD functionality for job listings. Next I want to work on authentication.